UNIT I

INTRODUCTION: Role of abstraction, Basic functional units of a computer, Von-Neumann model of computation, Moore's law, form Notion and perance- Data representation and basic operations.

ROLES OF ABSTARCTION

Abstraction plays a fundamental role in *Computer Organization and Architecture (COA)*, as it helps manage the complexity of modern computing systems. Here are the key roles of abstraction in COA:

1. Simplification of Complex Systems

- **Layered Approach**: Abstraction allows complex systems to be broken down into simpler, manageable layers. For example, high-level software can interact with abstracted hardware components like the CPU, memory, and I/O devices, without needing to know their detailed internal workings.
- **Focus on Key Features**: By abstracting away unnecessary details, abstraction helps focus on the key functionality, such as instruction execution or data storage, making it easier to understand and design systems.

2. Separation of Concerns

- **Hardware and Software Decoupling**: Abstraction separates the hardware implementation from the software layer. This enables software developers to write code without needing to understand or manage the hardware intricacies, promoting ease of development and reducing errors.
- **Independent Development**: Different subsystems, such as memory, CPU, and I/O, can be developed independently at various levels of abstraction, simplifying the development process and allowing specialization.

3. Encapsulation of Details

- **Hiding Complexity**: In COA, abstraction hides the low-level complexity of hardware, such as the intricate details of how memory is managed or how instructions are decoded and executed. For example, a programmer interacts with virtual memory or a CPU through high-level abstractions like system calls or APIs, without worrying about the physical memory layout or the instruction pipeline.
- **Transparent Operations**: Abstraction ensures that complex operations, like caching, memory management, and I/O handling, are done efficiently without the software needing to be aware of their implementation details.

4. Enabling Flexibility and Portability

- **Hardware Independence**: By abstracting hardware, software can be written to interact with a generalized interface, making it portable across different machines. For example, an operating system abstracts the details of the underlying hardware, enabling it to run on various CPU architectures (e.g., x86, ARM).
- **Adaptability**: Abstraction allows systems to be more easily adapted or reconfigured. Changes in one layer (such as the addition of a new processor or memory technology) can often be managed with minimal changes to the layers above it.

5. Facilitating Optimization and Efficiency

- **Performance Tuning**: Abstraction allows different levels of hardware and software to be optimized independently. For example, a high-level algorithm may be abstracted in a way that enables it to be

efficiently executed across various hardware configurations, such as utilizing SIMD (Single Instruction, Multiple Data) operations or multi-core processors without modifying the core algorithm.

- **Instruction Set Design**: Abstraction allows for a clean separation between the ISA (Instruction Set Architecture) and the hardware implementation, enabling the design of efficient processors without changing the software that runs on them.

6. Support for Concurrent and Parallel Processing

- **Concurrency Management**: Abstraction simplifies the management of parallelism. For example, higher-level abstractions like threads or processes allow software to take advantage of multi-core processors without needing to manage individual cores directly.
- **Pipelining and Parallelism**: Abstraction enables the design of techniques like pipelining, where multiple stages of instruction processing occur simultaneously. The software operates at a high level, unaware of the intricate scheduling and execution managed at the hardware level.

7. Improving Maintainability and Scalability

- **Code Reusability**: Abstraction allows the reuse of high-level code that interacts with lower-level hardware without worrying about the underlying details, making systems easier to maintain and scale.
- **Easier Upgrades**: Since abstraction hides the internal details, it becomes easier to upgrade components of the system (e.g., replacing the processor or adding new memory) without affecting the overall functionality of the software or the rest of the system.

8. Design Flexibility

- **Custom Hardware Design**: Abstraction allows different hardware architectures to be designed while maintaining a common interface to software. For example, RISC and CISC architectures provide different implementations of the instruction set, but both abstract those details from the software, ensuring portability across different designs.
- **Modular Design**: By abstracting the components of a system, designers can create modular systems that can be easily adapted or extended, such as swapping out memory types or integrating specialized hardware accelerators (e.g., GPUs, TPUs).

BASIC FUNCTIONAL UNITS OF A COMPUTER

A computer consists of five functionally independent main parts. They are

1. Input
2. Memory
3. arithmetic and logic
4. output and
5. control units

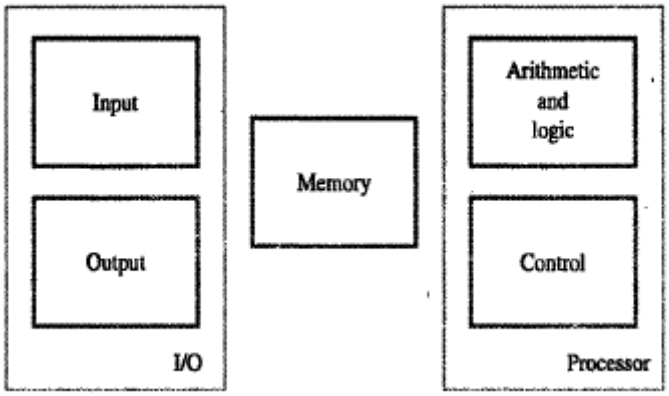


Fig.1.1 Basic functional units of a computer

INPUT UNIT

Computers accept coded information through input units, which read the data. The most well-known input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted over a cable to either the memory or the processor

MEMORY UNIT

The function of the memory unit is to store programs and data. There are two classes of storage, called primary and secondary.

Primary storage is a fast memory that operators at electronic speeds. Programs must be stored in the memory while they are being executed. The memory contains the large number of semi-conductor storage cells, each capable of storing one bit of information. These cells are rarely read or written as individual cells but instead are processed in groups of fixed size called words.

To provide easy access to any word in the memory, a distinct address is associated with each word location. Addresses are numbers that identify successive locations. The number of bits in each word is often refer to as the word length of the computer. Typical word length range from 16 to 64 bits.

Memory in which any location can be reached in a short and a fixed amount of timeafter specifying its address is called randam access memory(RAM) the time required to access one word is called memory access time.

The memory of the computer is normally implemented as a memory hierarchy of three or four levels of semi-conductor RAM units with different speeds and sizes. The small, fast RAM units are called Caches. The largest and slowest units is refer to as the main memory.

Primary storage is expensive so in additional secondary storage is used when large amount of data and many programs have to be stored, particularly for information i.e accessed infrequently. Secondary storage is also cheaper. The wide selection of secondary storage device is available, including magnetic disc and tapes and optical disc, CD-ROM.

ARITHMETIC AND LOGIC UNIT

Most computer operations are executed in the arithmetic and logic units(ALU) of the processor.

Arithmetic and logic operations – Multiplication, division, comparison of numbers etc.,

When operands are brought into the processor they are stored in high speed storage elements called Registers. Each Register can store one word of data

OUTPUT UNIT

The output units is the counter part of the input unit. Its function is to sent processed result to the outside world. Output units – Printer(inkjet printer, Laser printer).Graphics display provide both an input function and input function-I/O Unit.

CONTROL UNIT

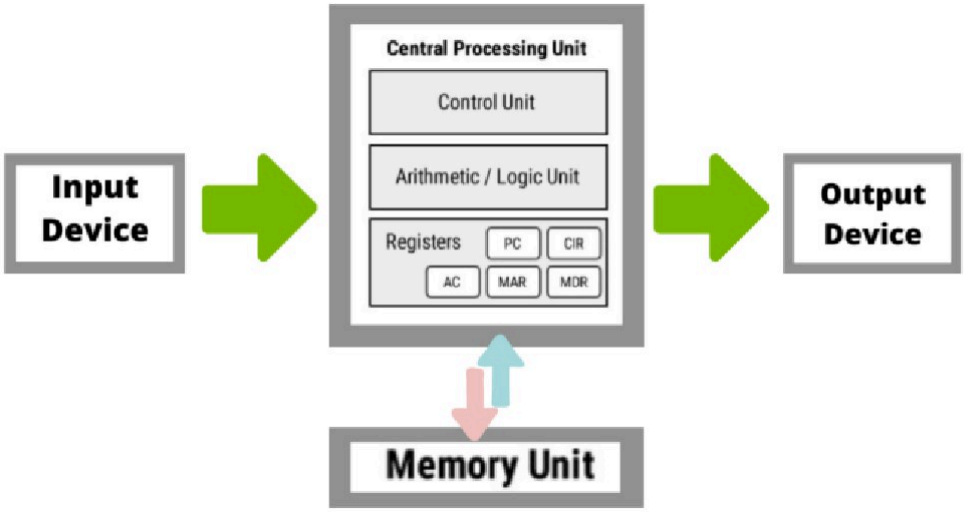
The control unit is effectively the node center that sends control signals to other units and senses their states. The actual timing signals that coverns the transfer or generated by the control circuit. The operation of a computer can be summarized as follows,

1. the computer accepts information in the form of programs and data through an input unit and stores it in the memory
2. information stored in the memory is fetched, under program control, into an arithmetic and logic unit, where it is processed.
3. Processed information leaves the computer through an output unit.
4. All activities inside the machines are directed by the control unit.

Von Neumann Architecture:

The Von-Neumann Architecture or Von-Neumann model is also known as **"Princeton Architecture"**. This architecture was published by the Mathematician **John Von Neumann** in **1945**.

Von Neumann architecture is the design upon which many general purpose computers are based. This architecture implemented the stored program concept in which the data and instructions are stored in the same memory. This architecture consists of a CPU(ALU, Registers, Control Unit), Memory and I/O unit.



Following are the components of Von Neumann Architecture:

1. CPU(Central processing unit)
   - CU(Control Unit)
   - ALU(Arithmetic and logic unit)
   - Registers
     - ✓ PC(Program Counter)
     - ✓ IR(Instruction Register)
     - ✓ AC(Accumulator)
     - ✓ MAR(Memory Address Register)
     - ✓ MDR(Memory Data Register)
2. BUSES
3. I/o Devices
4. Memory Unit

1. **<u>CPU:</u>** CPU acts as the brain of the computer and is responsible for the execution of instructions.
   a) **Control Unit:** A control unit (CU) handles all processor control signals. It directs all input and output flow, fetches code for instructions, and controls how data moves around the system.
   b) **Arithmetic and Logic Unit (ALU) :**
   The arithmetic logic unit is that part of the CPU that handles all the calculations the CPU may need, e.g. Addition, Subtraction, Comparisons. It performs Logical Operations, Bit Shifting Operations, and Arithmetic operations.
   c) **Registers**: A processor based on von Neumann architecture has five special registers which it uses for processing:

      - **Program counter (PC)** holds the memory address of the next instruction to be fetched from primary storage.
      - The **Memory Address Register(MAR)** holds the address of the current instruction that is to be fetched from memory, or the address in memory to which data is to be transferred.
      - The **Memory Data Register(MDR)** holds the contents found at the address held in MAR or data which is to be transferred to the primary storage.
      - The **Current Instruction Register(CIR)** holds the instruction that is currently being decoded and executed.
      - The **Accumulator** is a special purpose Register and is used by the ALU to hold the data being processed and the results of calculations.

2.BUSES :A bus is a subsystem that is used to connect computer components and transfer data between them. There are three types of BUSES

a) **Data Bus:** It carries data among the memory unit, the I/O devices, and the processor.
b) **Address Bus:** It carries the address of data (not the actual data) between memory and processor.
c) **Control Bus:** It carries control commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer.

3. **<u>I/o Devices</u>**:Program or data is read into main memory from the *input device* or secondary storage under the control of CPU input instruction. *Output devices* are used to output the information from a computer. If some results are evaluated by CPU and it is stored in the computer, then with the help of

output devices, we can present them to the user.

4. **Memory:**A memory unit is a collection of storage cells together with associated circuits needed to transfer information in and out of the storage. The memory stores binary information in groups of bits called words. The internal structure of a memory unit is specified by the number of words it contains and the number of bits in each word(2^Mx N, eg: 128KB).

There are two types of Primary Memory:

1)RAM: VOLATILE MEMORY or temporary Memory(to store the program in execution)

2)ROM: NON-VOLATILE MEMORY or permanent Memory(to store the booting program)

Moore's Law

Gordon Moore one of the co-founders of **Intel** corporation established the term "Moore's Law" in 1965, This law explains how the number of transistors on integrated circuits is increasing exponentially, which boosts computing capability and lowers prices. Moore's law has had a significant and far-reaching impact on technology, from the introduction of personal computers and smartphones to the advancement of artificial intelligence and Internet of Things(IoT) devices. it has accelerated development in sectors including telecommunications, healthcare, transportation, enabling organizations and people to accomplish things that were previously unthinkable.

Moore's Law is a key concept in *Computer Organization and Architecture (COA)*, describing the historical trend of exponential growth in the number of transistors on integrated circuits (ICs) over time, leading to more powerful and efficient processors. It was first observed by Gordon Moore, co-founder of Intel, in 1965, stating that the number of transistors on a microchip doubles approximately every two years, which leads to an increase in computing power and a decrease in cost per transistor.

Role of Moore's Law in COA:

1. **Increased Processor Performance**:
    - As the number of transistors on a chip increases, the processing power also increases, leading to faster execution of instructions and more sophisticated computations. For COA, this means more complex instruction sets can be executed in parallel, allowing for more efficient processing in modern CPUs.
2. **Miniaturization of Components**:
    - With Moore's Law, processors have become significantly smaller over time. Smaller components allow for more compact and energy-efficient hardware. This miniaturization leads to the development of devices like smartphones, laptops, and wearable technologies, all powered by increasingly advanced microprocessors.
    - The trend enables the development of more advanced hardware architectures, such as multi-core on-Chip (SoC) designs, which optimize performance while reducing physical size and

cessing:

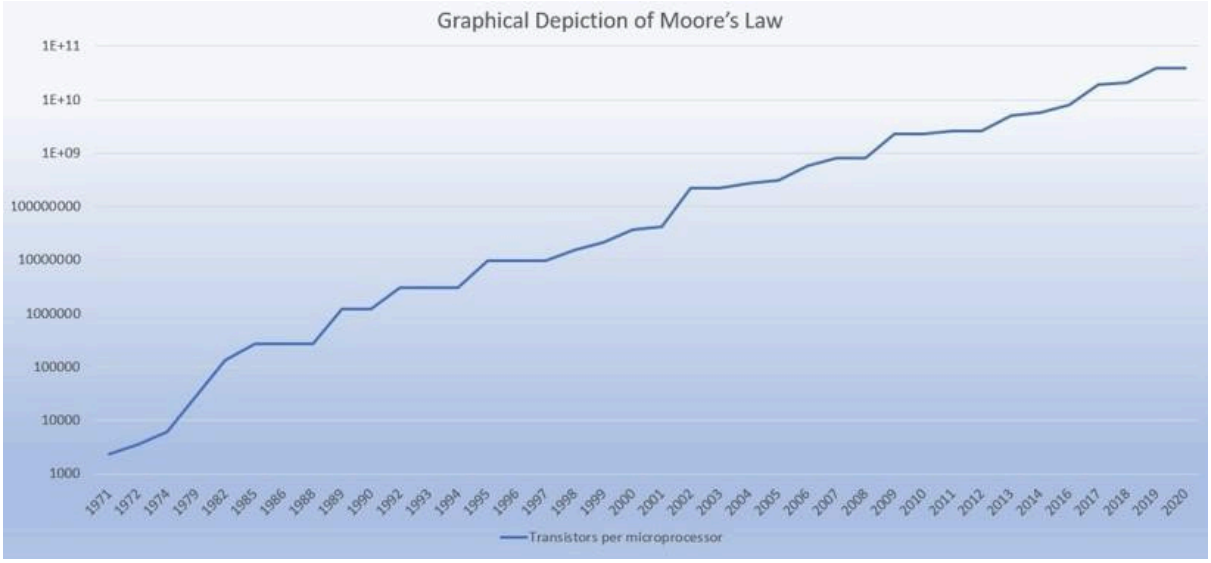
- As transistor density increases, manufacturers have turned to multi-core processors, allowing multiple tasks to be processed simultaneously. COA benefits from this shift, as multi-core architectures enable more efficient execution of parallel workloads, boosting overall system performance without a proportional increase in clock speed.
- For example, in modern CPUs, parallel processing via multiple cores is essential for handling complex tasks such as data processing, graphics rendering, and scientific simulations.

4. **Improved Power Efficiency**:
   - Moore's Law has not only led to higher performance but also increased the energy efficiency of processors. Smaller, more powerful transistors consume less power, leading to reduced heat generation and longer battery life in portable devices. This is especially important in mobile computing, where power consumption is a critical factor.
   - Energy efficiency allows for more complex systems to be built within the same power budget, which is particularly relevant for data centers and large-scale computing infrastructures.
5. **Advancements in Microarchitecture Design**:
   - The increase in transistor count has driven advancements in microarchitecture design. Modern microprocessors use sophisticated features like pipelining, branch prediction, out-of-order execution, and cache hierarchies, all made possible by the ever-growing number of transistors.
   - In COA, these architectural improvements allow for faster processing, better resource management, and more complex computing tasks to be handled effectively. For example, pipelining allows multiple instruction stages to be processed simultaneously, improving throughput.
6. **Impacts on Software Development**:
   - Moore's Law has also influenced software development, as more powerful processors allow for more complex and resource-demanding applications. Software can now leverage the increased processing power of modern hardware to perform advanced tasks such as machine learning, AI, and real-time data processing.
   - Software developers can optimize their applications to take advantage of the parallelism and improved processing power provided by multi-core CPUs, leading to more sophisticated and efficient software.
7. **Challenges and Limitations**:
   - While Moore's Law has held true for several decades, there are physical limitations to how small transistors can get. As we approach the atomic scale, further improvements in transistor density become increasingly difficult, leading to slower growth in processing power.
   - In response to this, researchers are exploring alternative computing paradigms like quantum computing, optical computing, and neuromorphic computing, which may offer ways to continue improving computational power beyond the limits of traditional transistor scaling.
8. **Future of Moore's Law**:

- Although the pace of Moore's Law has slowed in recent years, the principle still drives innovation in COA, especially in terms of optimizing architectures and exploring new materials and technologies (e.g., 3D stacking of chips, silicon photonics).
- Even as the number of transistors in a chip reaches a plateau, advances in other areas—such as AI-specific hardware, GPUs, and specialized accelerators—continue to push the boundaries of computational capabilities.

Moore's Law Definition

The exponential increase in the number of transistors on integrated circuits over time is referred to as Moore's law. According to this, a chip transistor count tends to double every two years or so, resulting in higher processing power and better performance.



Graph shows increase of transistors over the years in a microprocessor

Transistor Scaling

Transistor scaling, a crucial component of Moore's law, has proved essential in advancing technology in many different fields. Transistors act as switches that regulate the flow of electrical current within microchips, serving as the essential building blocks of electronic devices. As technology developed engineers concentrated on shrinking and packing transistors more closely together on a chip. A large increase in the number of transistors that could be put onto a single chip was made possible by this scaling process, Electronic devices were able to complete more complicated tasks faster due to the increased computing capacity brought on by the addition of transistors.

Impact on Computing Power

Moore's law has had nothing short of a transformational effect on computing power. The computing power of electronic devices has exponentially increased with each doubling of the number of transistors on a chip. The creation of sophisticated software applications, data processing capacities, and a wide range of technological advancements have all been spurred by this exponential growth. Computers can now process more operations per second because to the steady growth in transistor density, making computation faster and more effective. This has made it possible to develop complex software programs.

Challenges and Limitations

As transistors become closer to atomic scales, Moore's Law has recently encountered considerable difficulties and restrictions. The ongoing advancement of Mo0ore's law is hampered by a number of physical constraints as transistor size decreases. The quantum effects at such small scales are one of these restrictions. When electrons leak past barriers as a result of quantum tunneling, for example, transistor actions become inaccurate and unstable. As transistors get smaller this tendency gets stronger, making it more challenging to maintain steady, predictable behaviour.

Dissipation of heat presents another issue. Temperatures inside the microchip are raised as a result of increased heat generation as transistors are stacked closer together. Heat dissipation becomes a serious issue since it can impact dependability, reduce performance, and increase energy consumption.

NOTION AND PERFORMANCE

1. Notion

The term "notion" in COA refers to the fundamental concepts, principles, or models that define and guide the design of computer systems. These notions help in understanding how hardware and software interact to enable computation. Some of the key notions in COA include:

- **Instruction Set Architecture (ISA):** The ISA defines the set of instructions that a processor can execute. It is the abstraction layer between the hardware and software, allowing software to interact with hardware. The ISA also determines the data formats, addressing modes, and control instructions that the CPU can interpret.
- **Memory Hierarchy:** This notion refers to the different levels of memory, from the fastest (e.g., registers, cache) to the slowest (e.g., secondary storage like hard drives). It is essential for designing systems that optimize data access and storage.
- **Control Unit (CU):** The CU manages the execution of instructions by directing the flow of data and controlling the timing of operations within the CPU. It interprets instructions from memory and generates control signals that coordinate the rest of the system.
- **Pipelining:** Pipelining is a technique where multiple instruction stages are overlapped to increase throughput. This allows different stages of multiple instructions to be processed simultaneously, improving efficiency.
- **Parallelism:** The concept of performing multiple tasks or instructions simultaneously, either at the instruction level (ILP) or at the thread level (TLP). This notion is fundamental to improving computational speed and resource utilization.
- **Clock and Timing:** The clock governs the timing of operations in a system. Understanding clock cycles and how they influence instruction execution is critical in determining the overall performance of a computer.

2. Performance

Performance in COA refers to how efficiently a computer system executes tasks and processes data. Performance is a key aspect of computer architecture, as it dictates the responsiveness and capability of the system. It can be assessed in several ways:

- **Clock Speed:** The clock speed (measured in GHz) determines how quickly a processor can execute instructions. While higher clock speeds generally lead to better performance, they are not the only determinant of overall system speed.

- **Instruction Throughput:** This refers to the number of instructions a processor can execute per unit of time. Increasing throughput allows the system to handle more work in the same period.
- **Latency:** Latency is the time it takes to complete a single operation or instruction. Lower latency means tasks are completed faster, which is especially important for time-sensitive applications.
- **Pipelining and Superscalar Architecture:** These techniques improve performance by allowing multiple instructions to be processed simultaneously in different stages or by executing multiple instructions in parallel. Pipelining helps reduce idle CPU cycles and boosts throughput.
- **Memory Access Time:** The time it takes to access data from memory significantly affects performance. Systems with faster memory access times, such as those with multi-level caches, are typically faster.
- **Cache Hierarchy:** The use of cache memory helps to reduce the time spent accessing slower main memory. Efficient cache design, including multiple levels (L1, L2, and L3), improves system performance by keeping frequently accessed data closer to the CPU.
- **Parallel Processing:** Multi-core processors and the ability to execute multiple instructions or tasks in parallel lead to significant performance improvements. By utilizing more cores, tasks can be divided and processed simultaneously, improving overall throughput.
- **Energy Efficiency:** Modern processors are designed to balance performance with energy consumption. Lower power consumption while maintaining high performance is essential, particularly in mobile devices.
- **Benchmarks and Performance Metrics:** Performance is often measured using benchmarks that assess how quickly a system can complete specific tasks. Common performance metrics include:
    - **MIPS (Million Instructions Per Second):** Measures the number of instructions a processor can execute per second.
    - **FLOPS (Floating Point Operations Per Second):** A metric used to measure the performance of processors in scientific computing or complex mathematical calculations.
    - **IPC (Instructions Per Cycle):** Measures the number of instructions executed per clock cycle, reflecting how effectively the CPU uses its clock cycles.

Relationship Between Notion and Performance:

- The **notions** (such as ISA, pipelining, parallelism, and memory hierarchy) are the fundamental building blocks of how computer systems are designed. These concepts directly influence the **performance** of a system. For instance:
    - A well-designed **ISA** can simplify instruction execution, which improves throughput and reduces latency.
    - Techniques like **pipelining** and **superscalar architectures** are based on notions of overlapping and executing multiple instructions at once, directly enhancing performance.
    - The **memory hierarchy** notion affects how quickly the CPU can access data, influencing overall system speed.

. Data Types

- Computer does not understand human language. Any data, viz., letters, symbols, pictures, audio, videos,
- etc., fed to computer should be converted to machine language first. Computers represent data in the
- following three forms

Data / Number Representation and Arithmetic Operations

Integers

Consider an n-bit vector : B = bn−1 . . . b1b0 where bi = 0 or 1 for $0 \le i \le n - 1$.

$$V(B) = b_{n-1} \times 2^{n-1} + \cdots + b_1 \times 2^1 + b_0 \times 2^0$$

We need to represent both positive and negative numbers. Three systems are used for representing such numbers:

- Sign-and-magnitude
- 1"s-complement
- 2"s-complement

In all three systems, the leftmost bit is 0 for positive numbers and 1 for negative numbers.

B	Values represented		
$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0 1 1 1	+7	+7	+7
0 1 1 0	+6	+6	+6
0 1 0 1	+5	+5	+5
0 1 0 0	+4	+4	+4
0 0 1 1	+3	+3	+3
0 0 1 0	+2	+2	+2
0 0 0 1	+1	+1	+1
0 0 0 0	+0	+0	+0
1 0 0 0	−0	−7	−8
1 0 0 1	−1	−6	−7
1 0 1 0	−2	−5	−6
1 0 1 1	−3	−4	−5
1 1 0 0	−4	−3	−4
1 1 0 1	−5	−2	−3
1 1 1 0	−6	−1	−2
1 1 1 1	−7	−0	−1

In *1's-complement* representation, negative values are obtained by complementing each bit of the corresponding positive number. Thus, the representation for −3 is obtained by complementing each bit in the vector 0011 to yield 1100.

In the *2's-complement* system, forming the 2"s-complement of an n-bit number is done by subtracting the number from 2^n. Hence, the 2"s-complement of a number is obtained by adding 1 to the 1"s-complement of that number.

There are distinct representations for +0 and −0 in both the sign-and magnitude and 1"s-complement systems, but the 2"s-complement system has only one representation for 0.

Addition of Unsigned Integers

The sum of 1 and 1 is the 2-bit vector 10, which represents the value 2. We say that the *sum* is 0 and the *carry-out* is 1. We add bit pairs starting from the low-order (right) end of the bit vectors, propagating carries toward the high-order (left) end. The carry-out from a bit pair becomes the *carry-in* to the next bit pair to the left.

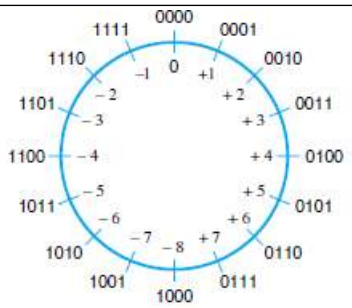
$$\begin{array}{r} 0 \\ +\ 0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 1 \\ +\ 0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 0 \\ +\ 1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 1 \\ \hline 1\,0 \end{array}$$

Carry-out

Addition and Subtraction of Signed Integers

The 2"s-complement system is the most efficient method for performing addition and subtraction operations.

Unsigned integers mod N is a circle with the values 0 through $N-1$. The decimal values 0 through 15 are represented by their 4-bit binary values 0000 through 1111.



(b) Mod 16 system for 2's-complement numbers

The operation (7 + 5) mod 16 yields the value 12. To perform this operation graphically, locate 7 (0111) on the outside of the circle and then move 5 units in the clockwise direction to arrive at the answer 12 (1100).
Similarly, (9 + 14) mod 16 = 7; this is modeled on the circle by locating 9 (1001) and moving 14 units in the clockwise direction past the zero position to arrive at the answer
7 (0111).
Apply the mod 16 addition technique to the example of adding +7 to −3. The 2"s-complement representation for these numbers is 0111 and 1101, respectively.

To *add* two numbers, add their *n*-bit representations, ignoring the carry-out bit from the most significant bit (MSB) position. The sum will be the algebraically correct value in 2"s-complement representation if the actual result is in the range -2^{n-1} through $+2^{n-1} - 1$.

To *subtract* two numbers *X* and *Y* , that is, to perform $X - Y$, form the 2"s-complement of *Y* , then add it to *X* using the *add* rule. Again, the result will be the algebraically correct value in 2"s-complement representation if the actual result is in the range -2^{n-1} through $+2^{n-1} - 1$.

	0010	(+2)	(b)	0100	(+4)
	+ 0011	(+3)		+ 1010	(−6)
	0101	(+5)		1110	(−2)
	1011	(−5)	(d)	0111	(+7)
	+ 1110	(−2)		+ 1101	(−3)
	1001	(−7)		0100	(+4)

Floating-Point Numbers

If we use a full word in a 32-bit word length computer to represent a signed integer in 2"s-complement representation, the range of values that can be represented is -2^{31} to $+2^{31} - 1$.

Since the position of the binary point in a floating-point number varies, it must be indicated explicitly in the representation. For example, in the familiar decimal scientific notation, numbers may be written as 6.0247×10^{23}, 3.7291×10^{-27}, -1.0341×10^{2}, -7.3000×10^{-14}. these numbers have been given to 5 *significant digits* of precision.

A binary floating-point number can be represented by:

- a sign for the number
- some significant bits
- a signed scale factor exponent for an implied base of 2

Character Representation

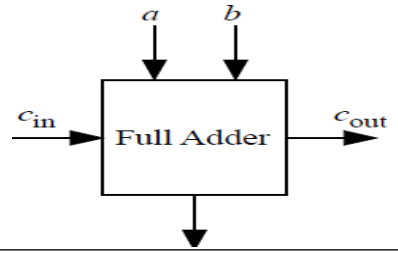
Bit positions	Bit positions 654							
3210	000	001	010	011	100	101	110	111
0000	NUL	DLE	SPACE	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	/	l	|
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

The most common encoding scheme for characters is ASCII (American Standard Code for Information Interchange). Alphanumeric characters, operators, punctuation symbols, and control characters are represented by 7-bit codes. It is convenient to use an 8-bit *byte* to represent and store a character.

The code occupies the low-order seven bits. The high-order bit is usually set to 0. This facilitates sorting operations on alphabetic and numeric data.

The low-order four bits of the ASCII codes for the decimal digits 0 to 9 are the first ten values of the binary number system.

This 4-bit encoding is referred to as the *binary-coded decimal* (BCD) code.

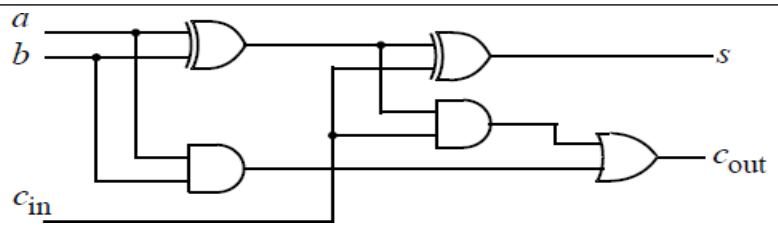


A one-bit full adder is a combinational circuit that forms the arithmetic sum of three bits. It consists of three inputs(a,b, and cin) and two outputs(s, and cout)_ as illustrated in Figure 1.

Table 1: Full adder truth table.

a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

The truth table of 1-bit full adder is given in the table



The gate implementation of 1-bit full adder is given in the figure

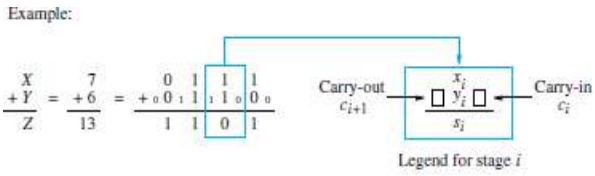


Figure 9.1 Logic specification for a stage of binary addition.

Ripple carry adder

A ripple carry adder is a digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adders connected in cascaded. with the carry output from each full adder connected to the carry input of the next full adder in the chain. Figure 3 shows the interconnection of four full adder (FA) circuits to provide a 4-bit ripple carry adder. Notice from Figure 3 that the input is from the right side because the first cell traditionally represents the least significant bit (LSB). Bits a0

and b0 _ in the figure represent the least significant bits of the numbers to be added. The sum output is represented by the bits s0 and s3

Ripple carry adder delays

In the ripple carry adder, the output is known after the carry generated by the previous stage is produced. Thus, the sum of the most significant bit is only available after the carry signal has rippled through the adder from the least significant stage to the most significant stage. As a result, the final sum and carry bits will be valid after a considerable delay.

Table 2 shows the delays for several CMOS gates assuming all gates are equally loaded for simplicity. All delays are normalized relative to the delay of a simple inverter. The table also shows the corresponding gate areas normalized to a simple minimum-area inverter. Note from the table that multiple-input gates have to use a different circuit technique compared to simple 2-input gates.

For an n-bit ripple carry adder the sum and carry bits of the most significant bit (MSB) are obtained after a normalized delay of

$$\text{Sum } s_{n-1} \text{ delay} = 4n+2 \quad (1)$$
$$\text{Carry } c_n \text{ delay} = 4n+3 \quad (2)$$

For a 32-bit processor, the carry chain normalized delay would be 131. The ripple carry adder can get very slow when many bits need to be added. In fact, the carry chain propagation delay is the determining factor in most microprocessor speeds.

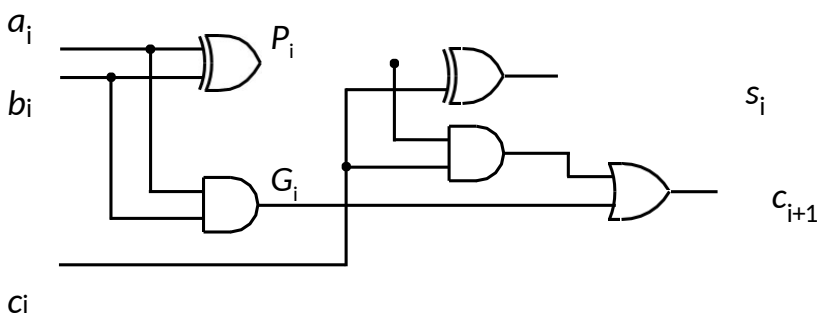
Carry lookahead adder (CLA)

The carry lookahead adder (CLA) solves the carry delay problem by calculating the carry signals in advance, based on the input signals. It is based on the fact that a carry signal will be generated in two cases:

(1) when both bits ai and bi are 1, or
(2) when one of the two bits is 1 and the carry-in is 1 .
Thus, one can write,

$$c_{i+1} = a_i.b_i + (a_i \oplus b_i).c_i \quad (3)$$
$$s_i = (a_i \oplus b_i) \oplus c_i \quad (4)$$

The above two equations can be written in terms of two new signals Pi and Gi , which are shown in Figure 4



Where Pi and Gi are called the carry generate and carry propagate terms, respectively. Notice that the generate and propagate terms only depend on the input bits and thus will be valid after one and two gate delay, respectively. If one uses the above expression to calculate the carry signals, one does not need to wait for the carry to ripple through all the previous stages to find its proper value. Let‟s apply this to a -bit adder to make it clear.

Notice that the carry-out bit, , of the last stage will be available after four delays: two gate de- lays to calculate the propagate signals and two delays as a result of the gates required to implement Equation 13.

$$c_{i+1} = G_i + P_i.c_i \quad (5)$$
$$s_i = P_i \oplus c_i \quad (6)$$

$$G_i = a_i.b_i \quad (7)$$
$$P_i = a_i \oplus b_i \quad (8)$$
$$(9)$$

Putting $i = 0, 1, 2, 3$ in Equation 5, we get

$$c_1 = G_0 + P_0.c_0 \quad (10)$$
$$c_2 = G_1 + P_1.G_0 + P_1.P_0.c_0 \quad (11)$$
$$c_3 = G_2 + P_2.G_1 + P_2.P_1.G_0 + P_2.P_1.P_0.c_0 \quad (12)$$
$$c_4 = G_3 + P_3.G_2 + P_3.P_2.G_1 + P_3.P_2.P_1.G_0 + P_3.P_2.P_1.P_0.c_0 \quad (13)$$

Figure 5 shows that a 4-bit CLA is built using gates to generate the and signals and a logic block to generate the carry out signals according to Equations 10–13.

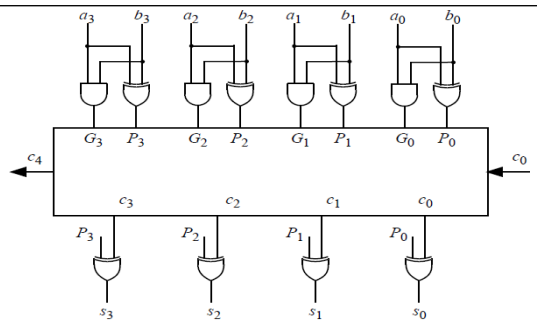
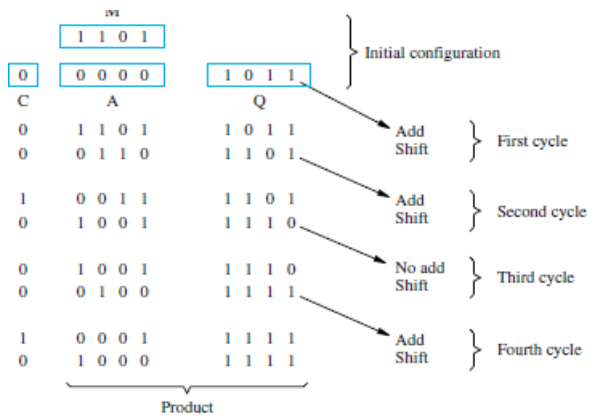


Figure 5: 4-Bit carry lookahead adder implementation detail.

The disadvantage of CLA is that the carry logic block gets very complicated for more than -bits. For that reason, CLAs are usually implemented as 4-bit modules and are used in a hierarchical structure to realize adders that have multiples of 4 -bits.

Shift – Add Multiplier

Multiplication is often defined as repeated additions. Thus, to calculate 11 × 23, you would start with 0 and add 11 to it 23 times.



(b) Multiplication example

In this, the 4 bit multiplier is stored in Q register, the 4 bit multiplicand is stored in register B and the register A is initially cleared to zero. The multiplication process starts with checking of the least significant bit of B whether it is 0 or 1.
If the B0 = 1, the number in the multiplicand (B) is added with the least significant bits of the A register and all bits of C, A and Q registers are shifted to the right one bit.
If the bit B0 = 0, the combined C and Q registers are shifted to the right by one bit without performing any addition. This process is repeated for n times for n bit numbers. This method of binary multiplication is called as parallel multiplier.

Consider the above figure in which the multiplier and multiplicand values are given as 1011 and 1101 which are loaded into the Q and A registers respectively.
Initially the register C is zero and hence the A register is zero, which stores the carry in addition.
Since the B0 =1, then the number in the B is added to the bits of A and produce the addition result as 1101, and the Q and A register are shifted their values one bit right so the new values during the first cycle are 0110 and 1101 respectively.
This process has to be repeated four times to perform the 4 bit multiplication. The final multiplication result will be available in the A and Q registers as 10001111

Booth Multiplier

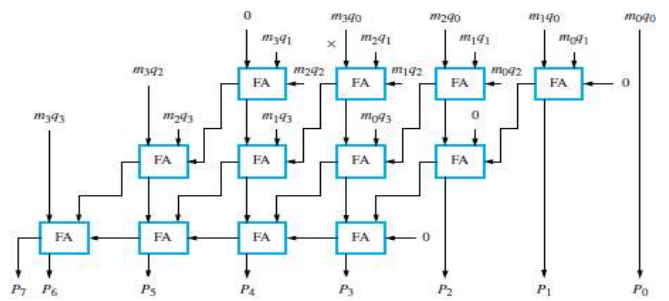
```
                    0 1 0 1 1 0 1
                    0 0+1+1+1+1 0
                    0 0 0 0 0 0 0
                  0 1 0 1 1 0 1
                0 1 0 1 1 0 1
              0 1 0 1 1 0 1
            0 1 0 1 1 0 1
          0 0 0 0 0 0 0
        0 0 0 0 0 0 0
  0 0 0 1 0 1 0 1 0 0 0 1 1 0

                    0 1 0 1 1 0 1
                    0+1 0 0 0−1 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0
  1 1 1 1 1 1 1 0 1 0 0 1 1   ← 2's complement of the multiplicand
  0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0
  0 0 0 1 0 1 1 0 1
  0 0 0 0 0 0 0 0
  0 0 0 1 0 1 0 1 0 0 0 1 1 0
```

The Booth algorithm generates a $2n$-bit product and treats both positive and negative 2"scomplement n-bit operands uniformly. In general, in the Booth algorithm, -1 times the shifted multiplicand is selected when moving from 0 to 1, and $+1$ times the shifted multiplicand is selected when moving from 1 to 0, as the multiplier is scanned from right to left.

```
  0 1 1 0 1   (+13)        0 1 1 0 1
× 1 1 0 1 0   (−6)   ⇒    0−1+1−1 0
                  0 0 0 0 0 0 0 0 0 0
                  1 1 1 1 1 0 0 1 1
                  0 0 0 0 1 1 0 1
                  1 1 1 0 0 1 1
                  0 0 0 0 0 0
                  1 1 1 0 1 1 0 0 1 0   (−78)
```

Carry-Save Addition



(b) Carry-save array

Multiplication requires the addition of several summands. A technique called *carry-save addition* (CSA) can be used to speed up the process.
This structure is in the form of the array in which the first row consists of just the AND gates that produce the four inputs m_3q_0, m_2q_0, m_1q_0, and m_0q_0.
Instead of letting the carries ripple along the rows, they can be "saved" and introduced into the next row, at the correct weighted positions.

- The data on which the instructions operate are not necessarily already stored in memory.
- Data need to be transferred between processor and outside world (disk, keyboard, etc.)
- I/O operations are essential, the way they are performed can have a significant effect on the performance of the computer.

  Program-Controlled I/O
  - Read in character input from a keyboard and produce character output on a display screen.

  Example
  - Rate of data transfer (keyboard, display, processor)
  - Difference in speed between processor and I/O device creates the need for mechanisms to synchronize the transfer of data.
  - A solution: on output, the processor sends the first character and then waits for a signal from the display that the character has been received. It then sends the second character. Input is sent from the keyboard in a similar way.

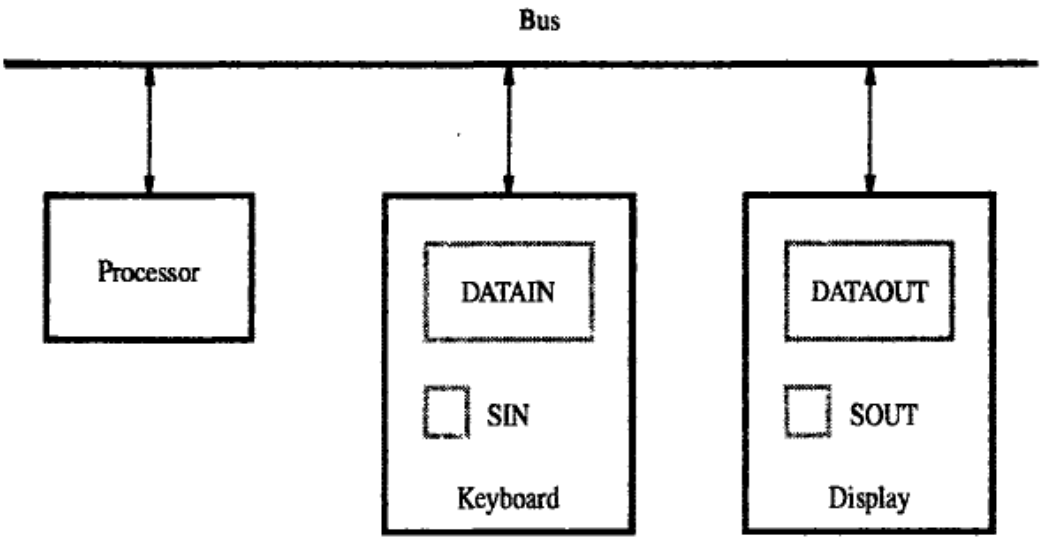


Fig. 1.9 Bus connection for processor, keyboard, and display.

- Machine instructions that can check the state of the status flags and transfer data:

```
READWAIT  Branch to READWAIT if SIN = 0
          Input from DATAIN to R1

WRITEWAIT Branch to WRITEWAIT if SOUT = 0
          Output from R1 to DATAOUT
```

- Memory-Mapped I/O – some memory address values are used to refer to peripheral device buffer registers. No special instructions are needed. Also use device status registers.

```
READWAIT  Testbit   #3, INSTATUS
          Branch=0  READWAIT
MoveByte  DATAIN, R1
```

- Assumption – the initial state of SIN is 0 and the initial state of SOUT is 1.
- Any drawback of this mechanism in terms of efficiency?
- Two wait loops→processor execution time is wasted

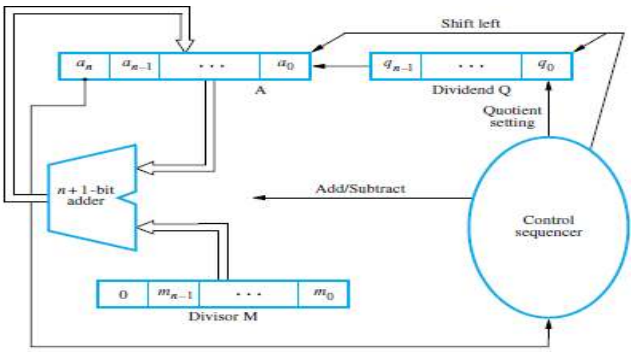
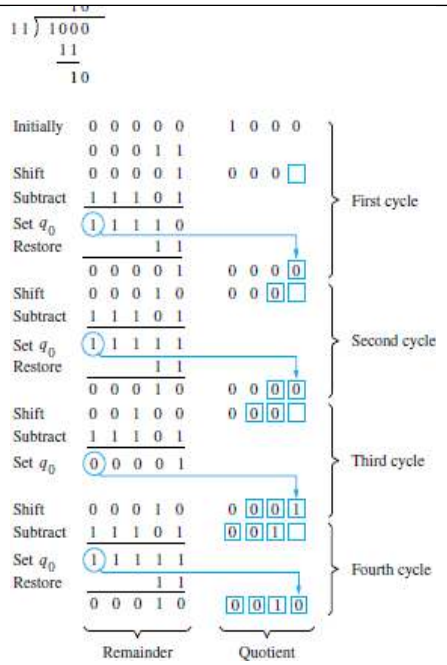


Figure 9.23 Circuit arrangement for binary division.



Restoring Division

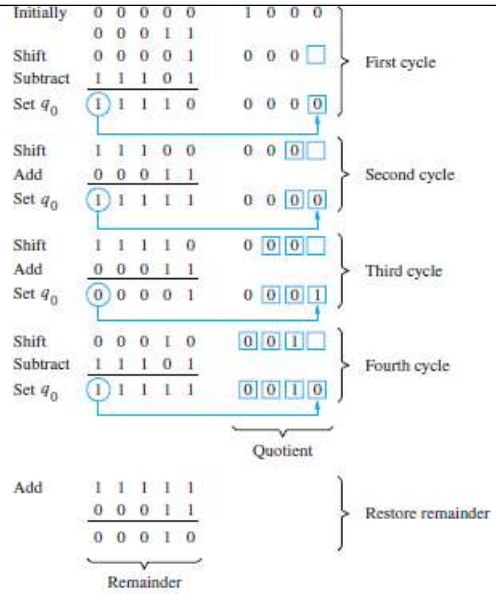


Figure 9.25 A non-restoring division example.

Non restoring division

An n-bit positive divisor is loaded into register M and an n-bit positive dividend is loaded into register Q at the start of the operation. Register A is set to 0. After the division is complete, the n-bit quotient is in register Q and the remainder is in register A.

The required subtractions are facilitated by using 2"s-complement arithmetic. The extra bit position at the left end of both A and M accommodates the sign bit during subtractions. The following algorithm performs restoring division.

Do the following three steps n times:

1. Shift A and Q left one bit position.
2. Subtract M from A, and place the answer back in A.
3. If the sign of A is 1, set q_0 to 0 and add M back to A (that is, restore A); otherwise, set q_0 to 1.

If A is positive, we shift left and subtract M, that is, we perform 2A− M. If Ais negative, we restore it by performing A+ M, and then we shift it left and subtract M.

This is equivalent to performing 2A+ M. The q_0 bit is appropriately set to 0 or 1 after the correct operation has been performed. following algorithm for *non-restoring division*.

Stage 1: Do the following two steps n times:

1. If the sign of A is 0, shift A and Q left one bit position and subtract M from A; otherwise, shift A and Q left and add M to A.
2. Now, if the sign of A is 0, set q_0 to 1; otherwise, set q_0 to 0.

Stage 2: If the sign of A is 1, add M to A.

Stage 2 is needed to leave the proper positive remainder in A after the n cycles of Stage 1.

Integer Division